REST API

Структура, данные и ошибки

Spring Boot: продолжение



Сегодня развиваем API задач

- Описываем операции и ожидаемые ответы.
- Разделяем HTTP, правила приложения и хранение.
- Принимаем данные через DTO и проверяем их.
- Возвращаем понятные статусы и ошибки.

После теории соберём всё в одном проекте.

Контракт нашего API

Операция	Запрос	Успех
Список задач	GET /tasks?done=false	200 + массив
Одна задача	GET /tasks/{id}	200 + задача
Создать	POST /tasks	201 + задача
Задать done	PATCH /tasks/{id}	200 + задача
Удалить	DELETE /tasks/{id}	204 без тела

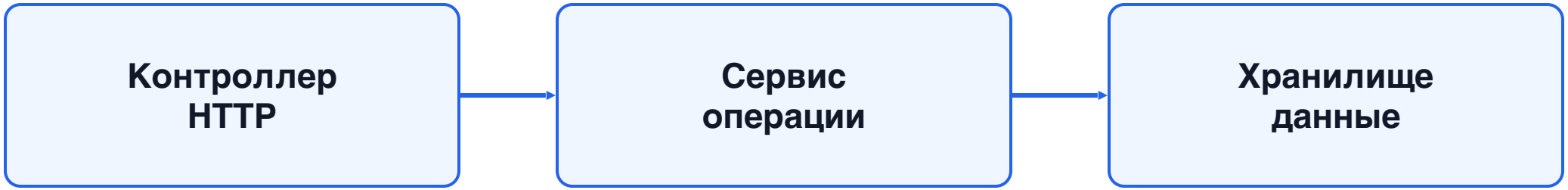
Пустой список возвращаем как 200 и [].

Ресурсы и HTTP-методы

- Задача является ресурсом с адресом /tasks/{id}.
- Метод выражает действие над ресурсом.
- JSON представляет данные задачи.
- Статус сообщает результат обработки.

Сначала договариваемся о поведении, затем пишем методы Java.

Разделение ответственности



Сервис вызывает хранилище, контроллер вызывает сервис.

Что находится в каждой части

Компонент	Пример ответственности
TaskController	Прочитать параметры и выбрать HTTP-ответ
TaskService	Создать задачу или сообщить, что её нет
TaskRepository	Сохранить и найти данные

@Repository обозначает компонент. Базу данных он сам не создаёт.

Модель задачи

Java • Task.java и пример использования

```
public record Task(
        long id,
        String title,
        boolean done) {
}

var task = new Task(1, "Git", false);
System.out.println(task.title());
```

record описывает данные.

id(), title(), done() читают компоненты.

Для изменения создадим новый Task.

Этот объект создаёт приложение, регистрировать его бином не требуется.

Хранилище скрывает способ хранения

Java • сигнатуры методов TaskRepository

```
public Optional<Task> findById(long id);
public List<Task> findAll();
public Task create(String title);
public Optional<Task> updateDone(
        long id, boolean done);
public boolean deleteById(long id);
```

Это список операций нашего хранилища.

Optional.empty() означает отсутствие задачи.

deleteById сообщает, была ли задача.

Контроллеру не требуется знать, какая коллекция хранит задачи.

Сервис получает хранилище

Java • фрагмент, импорты опущены

```
@Service
public class TaskService {
    private final TaskRepository repository;

    public TaskService(TaskRepository repository) {
        this.repository = repository;
    }

    public Task create(String title) {
        return repository.create(title);
    }
}
```

Конструктор получает зависимость.

Сервис описывает операцию приложения.

Детали хранения остаются в repository.

По мере роста правил меняется сервис, а контракт HTTP может сохраняться.

Контроллер получает сервис

Java • фрагмент, импорты опущены

```
@RestController
@RequestMapping("/tasks")
class TaskController {
    private final TaskService service;

    TaskController(TaskService service) {
        this.service = service;
    }
}
```

Общий путь задан на классе.

Методы добавим внутрь него.

Сервис получаем через конструктор.

В контроллере связываем HTTP с операциями приложения.

ID из пути: @PathVariable

Java • фрагмент, импорты опущены

```
@GetMapping("/{id}")
public TaskResponse get(
        @PathVariable("id") long id) {
    Task task = service.get(id);
    return TaskResponse.from(task);
}
```

GET /tasks/42

id получит значение 42.

/tasks/abc не преобразуется в long и даст 400.

Корректный ID ещё не означает, что задача существует.

Фильтр из query: @RequestParam

Java • фрагмент, импорты опущены

```
@GetMapping
public List<TaskResponse> list(
        @RequestParam(name = "done",
                      required = false) Boolean done) {
    return service.list(done).stream()
        .map(TaskResponse::from)
        .toList();
}
```

GET /tasks → done = null

?done=false → false

?done=true → true

Boolean позволяет отличить отсутствие фильтра от false.

DTO запроса и DTO ответа

Java • два отдельных DTO

```
public record CreateTaskRequest(String title) {}

public record TaskResponse(
        long id,
        String title,
        boolean done) {}
```

DTO = Data Transfer Object.

Запрос содержит то, что задаёт клиент.

Ответ содержит то, что возвращает сервер.

Клиент передаёт title. ID назначает сервер.

Преобразование модели в ответ

Java • фрагмент, импорты опущены

```
public record TaskResponse(
        long id, String title, boolean done) {

    public static TaskResponse from(Task task) {
        return new TaskResponse(
            task.id(), task.title(), task.done());
    }
}
```

Метод from копирует нужные данные.

Можно явно выбрать поля ответа.

JSON затем сформирует Jackson.

Изменения внутренней модели не обязаны менять публичный ответ.

Создание: @RequestBody и 201

Java • фрагмент, импорты опущены

```
@PostMapping
public ResponseEntity<TaskResponse> create(
        @RequestBody CreateTaskRequest request) {
    Task task = service.create(request.title());
    URI location = URI.create("/tasks/" + task.id());
    return ResponseEntity.created(location)
        .body(TaskResponse.from(task));
}
```

Jackson читает JSON в DTO.

Сервис создаёт задачу.

created задаёт 201 и Location.

ResponseEntity позволяет задать статус, заголовки и тело.

Ответ на создание задачи

HTTP/1.1 • основные строки ответа

```
HTTP/1.1 201 Created
Location: /tasks/1
Content-Type: application/json

{"id":1,"title":"Git","done":false}
```

201 сообщает об успешном создании.

Location указывает на новую задачу.

Тело содержит её данные.

Следующий GET /tasks/1 должен вернуть созданную задачу.

Изменение признака done

Java • фрагмент, импорты опущены

```
public record UpdateTaskRequest(
        @NotNull Boolean done) {}

@PatchMapping("/{id}")
public TaskResponse update(
        @PathVariable("id") long id,
        @Valid @RequestBody UpdateTaskRequest request) {
    return TaskResponse.from(
        service.updateDone(id, request.done()));
}
```

PATCH /tasks/1

Тело: {"done":true}

Меняем только done.

Неизвестная задача даст 404.

Это наш явно описанный формат PATCH-запроса.

Удаление с ответом 204

Java • фрагмент, импорты опущены

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(
        @PathVariable("id") long id) {
    service.delete(id);
    return ResponseEntity.noContent().build();
}
```

DELETE /tasks/1

Удалили → 204.

Задачи нет → 404 по нашему контракту.

Ответ 204 не содержит тела.

Подключение валидации

pom.xml • Spring Boot 4.x

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-validation
  </artifactId>
</dependency>
```

Добавляем к веб-проекту.

Версию согласует Spring Boot.

Аннотации импортируем из jakarta.validation.

Валидации нужен провайдер, одних аннотаций недостаточно.

Ограничения на входные данные

Java • фрагмент, импорты опущены

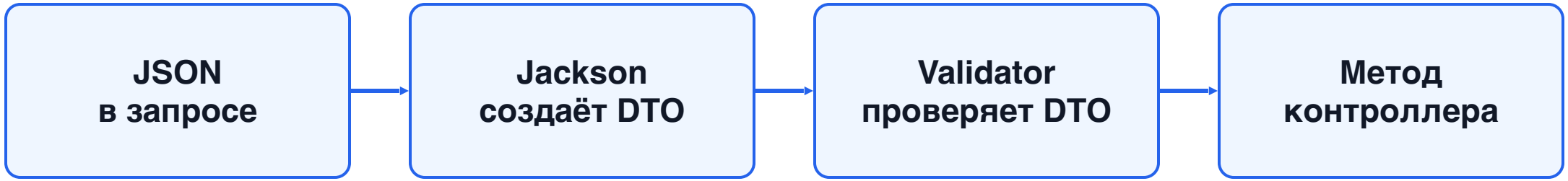
```
public record CreateTaskRequest(
        @NotBlank
        @Size(max = 100)
        String title) {}
```

@NotBlank отклоняет null, пустую строку и одни пробелы.

@Size ограничивает длину.

Ограничения описывают допустимые данные запроса.

Что запускает @Valid



При ошибке валидации наш метод создания не вызывается.

POST после добавления проверки

Java • фрагмент, импорты опущены

```
@PostMapping
public ResponseEntity<TaskResponse> create(
        @Valid @RequestBody CreateTaskRequest request) {
    Task task = service.create(request.title());
    URI location = URI.create("/tasks/" + task.id());
    return ResponseEntity.created(location)
        .body(TaskResponse.from(task));
}
```

Изменили параметр метода.

Некорректное название даст 400.

Сервис получит проверенное название.

Теперь ограничения DTO участвуют в обработке HTTP-запроса.

Ошибки возникают на разных этапах

Ситуация	Где обнаруживаем	Ответ
Сломан JSON	Чтение тела	400
title состоит из пробелов	Валидация DTO	400
id = abc	Преобразование параметра	400
Задачи с ID нет	Сервис / хранилище	404
Неожиданный сбой	Сервер	500

Входные ограничения и правила операции проверяются в разных местах.

Сервис сообщает об отсутствии задачи

Java • метод сервиса и отдельный класс исключения

```
public Task get(long id) {
    return repository.findById(id)
        .orElseThrow(() ->
            new TaskNotFoundException(id));
}

public class TaskNotFoundException
        extends RuntimeException {
    public TaskNotFoundException(long id) {
        super("Task " + id + " not found");
    }
}
```

get находится в TaskService.

Исключение описывает причину.

HTTP-статус назначим отдельно.

Операция сообщает, что задача не найдена. Веб-слой выбирает 404.

Общий обработчик ошибок

Java • фрагмент, импорты опущены

```
@RestControllerAdvice
class ApiErrors extends ResponseEntityExceptionHandler {
    @ExceptionHandler(TaskNotFoundException.class)
    ProblemDetail notFound(TaskNotFoundException ex) {
        return ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
    }
}
```

Advice обрабатывает ошибки контроллеров.

Наше исключение превращается в 404.

Базовый класс обрабатывает стандартные ошибки MVC.

Одно место связывает исключения приложения с HTTP-ответами.

ProblemDetail: формат ошибки

HTTP/1.1 • основные строки и тело ошибки

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "Task 42 not found",
  "instance": "/tasks/42"
}
```

status соответствует HTTP-статусу.

detail объясняет конкретную проблему.

instance связывает её с запросом.

Стандартный формат позволяет одинаково читать разные ошибки.

Единый формат ошибок MVC

- Базовый ResponseEntityExceptionHandler формирует ProblemDetail.
- Он обрабатывает стандартные ошибки чтения и валидации.
- При необходимости добавляем список ошибок полей.
- Клиенту не отправляем stack trace и внутренние детали.

Общий формат не означает одинаковый статус для всех ошибок.

Общие бины и данные конкретного запроса

- Контроллер и сервис обычно существуют в одном экземпляре.
- Параметры запроса держим в аргументах и локальных переменных.
- Поля с зависимостями можно сделать final.
- Общее изменяемое хранилище требует защиты от конкурентных вызовов.

final фиксирует ссылку, но не делает коллекцию потокобезопасной.

Что означает хранение в памяти

- Задачи живут внутри запущенного процесса.
- После перезапуска данные исчезают.
- Два экземпляра приложения имеют разные наборы задач.
- Сервис работает через операции хранилища.

Следующий шаг курса: постоянное хранение и база данных.

Проверка API через curl

Терминал • Bash / zsh

```
curl -i http://localhost:8080/tasks

curl -i -H "Content-Type: application/json" \
  --data-binary @task.json \
  http://localhost:8080/tasks

curl -i -X DELETE http://localhost:8080/tasks/1
```

task.json:
{"title":"Git"}

GET → 200
POST → 201
DELETE → 204

Проверяем статус, заголовки и тело ответа.

Какие сценарии обязательно проверить

Запрос / данные	Ожидаемое поведение
GET /tasks при пустом хранилище	200 и []
POST с корректным title	201, Location и задача
POST с title = " "	400 и ProblemDetail
GET /tasks/999, если задачи нет	404 и ProblemDetail
DELETE существующей задачи	204, затем GET по ID даёт 404

Работающий API проверяют и на успешных, и на ошибочных запросах.

Где должна находиться эта логика?

- Прочитать id из URL.
- Отклонить пустое название задачи.
- Сообщить, что задача не найдена.
- Найти запись в коллекции.

Контроллер, валидация DTO, сервис или хранилище?

Разбор: у каждой проверки своё место

Действие	Место в нашем приложении
Прочитать id	MVC и параметр контроллера
Проверить пустое название	Ограничения DTO и @Valid
Решить, что отсутствие задачи — ошибка	Сервис
Найти запись в коллекции	Хранилище

Контроллер знает HTTP. Сервис знает операцию. Хранилище знает данные.

Материалы к занятию

- Spring MVC: маршруты и параметры методов.
- Bean Validation: ограничения и @Valid.
- Spring MVC: ResponseEntity и обработка исключений.
- RFC 9457: Problem Details for HTTP APIs.

Ссылки на официальные документы находятся в заметках.

Демонстрация: развиваем API задач

Операции и DTO, затем валидация и ошибки
В конце проверяем всё через curl